Лекция 8. Поток минимальной стоимости.

1. Введение

До сих мы рассматривали две разные задачи на графах: поиск максимального потока (сколько единиц товара можно доставить от истока s к стоку t) и поиск кратчайшего пути (как оптимальнее всего добраться из точки А в точку Б). В реальной жизни эти задачи часто возникают вместе. У нас есть сеть труб (дорог), каждая из которых имеет не только пропускную способность (сколько литров в час может пропустить), но и стоимость прокачки одной единицы жидкости (цена за литр, время в пути, износ дороги).

Постановка задачи:

Дан ориентированный граф $G = (V, E)$, исток s, сток t. Для каждого ребра (u, v) заданы две характеристики:

1. **Пропускная способность:** $c(u, v) \geq 0$.
2. **Стоимость (вес):** $w(u, v)$. Обычно это стоимость прокачки **одной единицы** потока по этому ребру.

Необходимо найти поток f величины $|f| = F$ (или максимально возможной), который минимизирует суммарную стоимость:

$$Cost(f) = \sum_{(u,v)\in E} f(u, v) \cdot w(u, v).$$

В рамках лекции мы рассмотрим классический подход к решению этой задачи, основанный на комбинации идей поиска кратчайших путей и максимального потока.

2. Базовые понятия и свойства

2.1 Стоимость потока и остаточная сеть

Определение стоимости потока линейно: если мы пускаем 5 единиц по ребру с ценой 2, это стоит 10.

Ключевую роль снова играет понятие **остаточной сети**, но теперь мы должны учитывать не только пропускную способность, но и стоимость обратных рёбер.

Если по ребру (u, v) течёт поток $f(u, v)$ и его пропускная способность $c(u, v)$, то в остаточной сети:

1. Прямое ребро (u, v) сохраняется с остаточной пропускной способностью $c_f(u, v) = c(u, v) - f(u, v)$ и **прежней стоимостью** $w(u, v)$.
2. Появляется **обратное ребро** (v, u) с остаточной пропускной способностью $f(u, v)$ и **стоимостью** $-w(u, v)$.

Почему стоимость обратного ребра отрицательная?

Представьте, что мы уже отправили 3 единицы по дороге $u \to v$ и заплатили за это. Если позже мы решим уменьшить поток на этом ребре (т.е. «отменить» отправку), это эквивалентно тому, что мы получаем деньги обратно. В терминах остаточной сети: пустить 1 единицу по обратному ребру (v, u) – значит уменьшить поток в прямом направлении, что уменьшает общую стоимость на $w(u, v)$.

2.2 Декомпозиция потока

Как и в обычных потоках, любой поток можно разложить в сумму потоков вдоль путей из s в t и циклов. Это свойство позволит нам строить решение итеративно.

3. Наивный подход: последовательное увеличение

Самая простая идея (аналог Форда-Фалкерсона) – увеличивать поток на 1 единицу за шаг, каждый раз выбирая самый дешёвый путь.

Базовый алгоритм (для поиска потока величины F):

1. Инициализировать поток $f = 0$.
2. **Повторять F раз (или пока есть путь):**
3. В остаточной сети G_f найти путь из s в t с **минимальной суммарной стоимостью**.
4. Если путь не найден – остановиться (нельзя доставить требуемый объём).
5. Пустить по этому пути 1 единицу потока (или минимальную пропускную способность на пути, если мы не хотим ходить по одному), обновить остаточную сеть.

Псевдокод (базовый уровень):

```
function min_cost_flow_basic(graph, s, t, required_flow):
  flow = 0
  cost = 0
  # Предполагаем, что в графе изначально нет отрицательных циклов
  while flow < required_flow:
    # 1. Ищем путь минимальной стоимости в остаточной сети
    # Используем Bellman-Ford, т.к. могут быть отрицательные веса (обратные ребра)
    dist, parent = bellman_ford(graph_residual, s)

    if dist[t] == infinity:
      break # Пути нет, требуемый поток недостижим

    # 2. Определяем, сколько можем пустить (обычно 1, но можно больше)
    # Для простоты будем пускать 1 единицу, но правильнее искать минимум на пути
```

```
        delta = 1
        # 3. Обновляем поток и стоимость
        v = t
        while v != s:
            u = parent[v]
            f[u][v] += delta
            f[v][u] -= delta
            cost += delta * w(u, v) # Учитываем стоимость
            # Обновляем остаточную сеть (уменьшаем capacity прямого, увеличиваем обратного)
            v = u

        flow += delta

    return flow, cost
```

Анализ и проблемы:

- **Проблема 1. Отрицательные рёбра.** После первого же шага в остаточной сети появляются обратные рёбра с отрицательным весом. Дейкстра не работает. Приходится использовать медленный Форд-Беллман ($O(V \cdot E)$) на каждой итерации.
- **Проблема 2. Количество итераций.** Если мы пускаем по 1 единице, число итераций равно величине потока, которая может быть огромной.
- **Проблема 3. Отрицательные циклы.** В процессе работы в остаточной сети могут появляться отрицательные циклы. Наличие такого цикла означает, что текущий поток не оптимален по стоимости (даже если мы его только что нашли)! Мы можем гонять поток по этому циклу, увеличивая его величину (ноль не меняется, т.к. это цикл), но уменьшая общую стоимость. Это ломает логику «последовательного увеличения».

4. Метод потенциалов (кратчайший путь + Дейкстра)

Классическое решение – избавиться от отрицательных весов в остаточной сети с помощью потенциалов, чтобы можно было использовать Дейкстру. Идея заимствована из алгоритма Джонсона для поиска кратчайших путей между всеми парами вершин.

4.1 Определение потенциалов

Введём для каждой вершины v потенциал $\phi(v)$. Переопределим стоимость ребра (u, v) как:

$$w'(u, v) = w(u, v) + \phi(u) - \phi(v).$$

Свойство. Для любого цикла сумма исходных весов равна сумме приведённых весов, так как потенциалы сокращаются. Следовательно, наличие отрицательных циклов инвариантно относительно такого преобразования.

Как выбрать потенциалы?

Запустим один раз Форда-Беллмана из истока s в самом начале. Пусть $\phi(v)$ – это расстояние (минимальная стоимость пути) от s до v в исходном графе. Тогда для любого ребра (u, v) выполняется неравенство треугольника:

$$\phi(v) \leq \phi(u) + w(u, v).$$

Отсюда следует, что:

$$w'(u, v) = w(u, v) + \phi(u) - \phi(v) \geq 0.$$

Все приведённые стоимости становятся неотрицательными!

4.2 Поддержание потенциалов при изменениях графа

Главный фокус в том, чтобы поддерживать потенциалы актуальными при изменениях остаточной сети. После того как мы пустили поток по найденному пути, в графе появляются обратные рёбра. Не испортят ли они неотрицательность?

Пусть мы нашли кратчайший путь из s в t в графе с неотрицательными весами (с помощью Дейкстры). В этом пути:

- Для каждого прямого ребра (u, v) на пути, $w'(u, v) = 0$. Почему? Потому что это ребро лежит на кратчайшем пути, значит, $\phi(v) = \phi(u) + w(u, v)$, откуда $w'(u, v) = 0$.
- Когда мы пускаем поток по этому пути, в остаточной сети появляются **обратные рёбра** (v, u). Их приведённая стоимость будет:

$$w'(v, u) = w(v, u) + \phi(v) - \phi(u).$$

Но $w(v, u) = -w(u, v)$ (обратное ребро). Подставляем:

$$w'(v, u) = -w(u, v) + \phi(v) - \phi(u) = -\big(w(u, v) + \phi(u) - \phi(v)\big) = \\ = -w'(u, v) = 0.$$

Таким образом, **все новые рёбра, появляющиеся после увеличения потока, также имеют нулевую приведённую стоимость**. Неотрицательность весов сохраняется! На следующей итерации мы можем снова смело запускать Дейкстру.

Нам лишь нужно не забывать обновлять «истинные» расстояния. После того как Дейкстра нашла новый кратчайший путь, мы должны скорректировать потенциалы, добавив к ним найденные расстояния, чтобы поддерживать свойство неотрицательности для следующих итераций.

Псевдокод (алгоритм последовательного кратчайшего пути):

```
function min_cost_flow(graph, s, t, required_flow):
  n = |V|
  # Потенциалы. Инициализируем расстояниями от s (запускаем Форда-Беллмана 1 раз)
  phi = bellman_ford(original_graph, s) # O(V*E)
```

```
    if phi содержит отрицательный цикл, достижимый из s: # Задача не имеет смысла
        return error

    flow = 0
    cost = 0
    # Структуры для хранения потока f[u][v]

    while flow < required_flow:
        # 1. Запускаем Дейкстру в остаточной сети с приведенными весами w'(u,v) = w(u,v) + phi[u] - phi[v]
        # Используем priority queue
        dist, parent_edge = dijkstra(graph_residual, s, phi)

        if dist[t] == infinity:
            break # Требуемый поток недостижим

        # 2. Обновляем потенциалы для всех вершин
        for v in V:
            if dist[v] < infinity:
                phi[v] += dist[v]

        # 3. Находим минимальную пропускную способность на пути (в исходных единицах)
        delta = INF
        v = t
        while v != s:
            u = parent_edge[v].from
            # Смотрим на остаточную пропускную способность ребра (u,v)
            delta = min(delta, residual_capacity(u, v))
            v = u

        # 4. Обновляем поток, стоимость и остаточную сеть
        v = t
        while v != s:
            u = parent_edge[v].from
            f[u][v] += delta
            f[v][u] -= delta
            # Стоимость считаем по исходным весам!
            cost += delta * w(u, v)
            # Обновляем residual_capacity (уменьшаем c_f[u][v], увеличиваем c_f[v][u])
            v = u

        flow += delta
```

return flow, cost

Сложность. $O\big(F \cdot (E \cdot \log V)\big)$, где F – величина потока. Для максимального потока F может быть большой, но на практике часто используется вариация, где мы ищем не кратчайший путь, а сразу «блокирующий» поток в слоистой сети (аналог Диница, но со стоимостью), что даёт $O(|f| \cdot V \cdot E)$. Однако метод потенциалов с Дейкстрой – самый популярный для реализации «из коробки».

5. Обобщение: Циркуляция минимальной стоимости

Задача становится ещё интереснее, если у нас нет истока и стока, а есть просто граф, и мы хотим найти циркуляцию (поток, у которого в каждой вершине сумма входящего равна сумме исходящего), удовлетворяющую нижним и верхним границам на рёбрах, с минимальной стоимостью. Это обобщение задачи о потоках.

Один из способов свести её к уже решённой задаче:

1. Удовлетворить нижние границы. Пускаем по каждому ребру поток, равный нижней границе. Это нарушает баланс в вершинах (образуется дисбаланс $d[v]$).

2. Строим новую сеть, где истоком соединяемся с вершинами, у которых положительный дисбаланс (нужно «вытолкнуть» лишнее), а вершины с отрицательным дисбалансом соединяем со стоком («втянуть» недостающее).

3. Решаем в этой сети задачу о потоке минимальной стоимости, где стоимость рёбер соответствует исходной, а рёбра от/до нового истока/стока имеют нулевую стоимость.

Этот подход (сначала насытить все «хорошие» отрицательные рёбра, а потом балансировать остатки через поиск кратчайших путей) лежит в основе многих практических реализаций.

6. Сравнение подходов

Метод	**Сложность**	**Особенности**
Последовательное увелич. (Bellman-Ford)	$O(F \cdot V \cdot E)$	Прост для понимания, но очень медленный на больших потоках.
Последовательное увелич. (Дейкстра + потенциалы)	$O(F \cdot E \cdot \log V)$	Стандартный выбор для умеренных F. Гарантирует целочисленность.
Масштабирование пропускной способности	$O(E \cdot \log V \cdot \log C)$	Работает с бинарным представлением C. Очень быстр на практике, особенно когда C велико.

Алгоритм минимального среднего цикла	Полиномиальный	Больше теоретическое значение, сложен в реализации, но гарантирует полиномиальное время без привязки к F или C.